



Procesamiento de Señales

S2

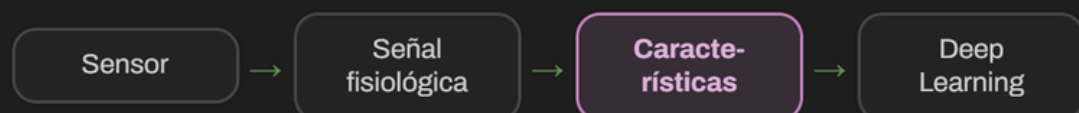
Resumen de clase · Señales Biomédicas ·
UPCH

entorno → en Drive: clic derecho → *Más*, o
Conectar más aplicaciones → “Google
Colaboratory” → **Conectar**.

RESUMEN

Filtros · dominio z

1.1 Extracción de características



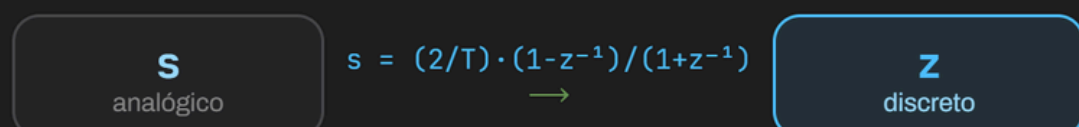
El paso más importante de la cadena.

1.2 Filtro = respuesta al impulso

 $\delta[n]$ · delta de Kronecker $h[n]$ = el filtro mismo

Cada capacitor o bobina de un circuito actúa como filtro; los diagramas de fase y magnitud dan su ancho de banda.

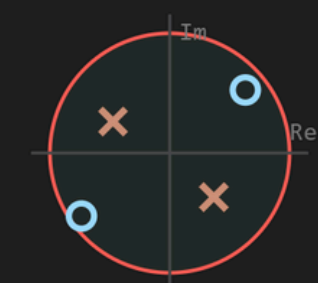
1.3 Transformada bilineal



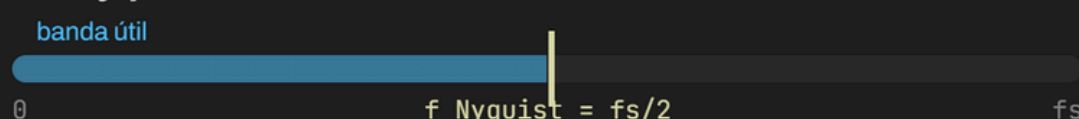
convolución → multiplicación

todo LTI debe ser **acotado**

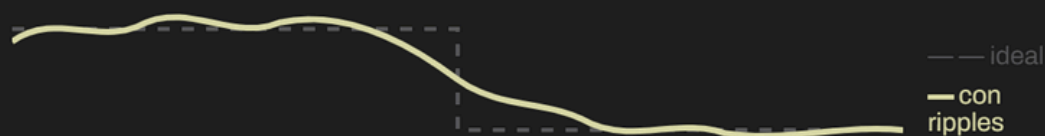
1.4 Polos y ceros

X **polos** dentro del círculo → estableX **polos** amplifican / resuenanO **ceros** atenúan frecuenciasO **ceros** en cualquier posición

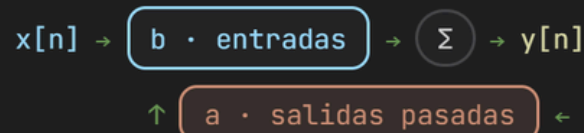
1.5 Nyquist: dos cosas distintas

Teorema: $fs \geq 2 \cdot f_{\text{máx}}$. Frecuencia: la mitad de fs .

1.6 FIR · fenómeno de Gibbs

Suma de muestras desplazadas (n desplazamientos $\approx$ derivada de orden n). El método de ventana trunca la respuesta ideal y aparecen los **ripples**.

1.7 IIR · realimentación

La realimentación agrega riesgo de **inestabilidad**: revisa polos y ceros antes de aplicarlo.

PARTE 1

Teoría · señales sintéticas

01 · senoidal.py

GENERAR

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 f, A, phi = 100, 1, 0 # señal de 100 Hz
5 w = 2*np.pi*f
6 fs = 10*f # freq. de muestreo
7 ts = 1/fs
8 n = np.arange(1000) # n° muestras
9 t = n*ts
10
11 x = A*np.sin(w*t + phi)
12 plt.plot(x); plt.xlim(0,50); plt.show()
```

$$fs \geq 2 \cdot f_{\text{máx}}$$

Regla de muestreo. Al menos el doble de la frecuencia máxima. A mayor fs , mejor se reconstruye la senoide.

02 · espectro.py

DFT → FFT

```
1 plt.magnitude_spectrum(x, scale="dB", Fs=fs)
2 plt.show()
```

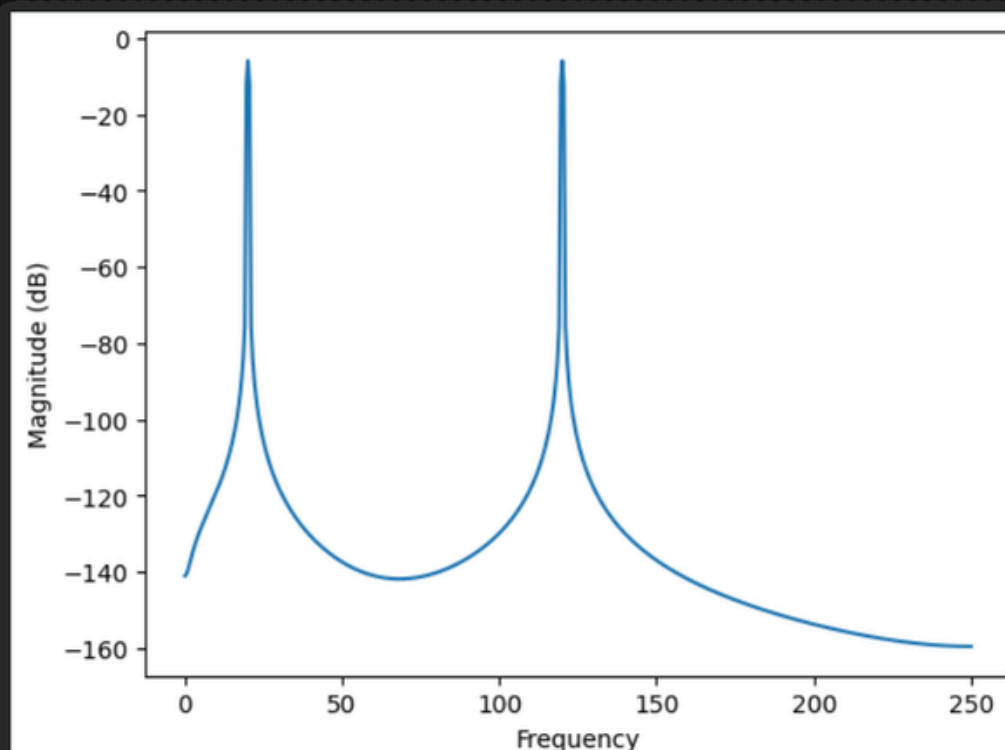
La DFT va de la muestra 0 a la última; la **FFT** la optimiza. Primer paso ante cualquier señal: **ver su espectro** — cada pico es una componente frecuencial.

03 · filtro.py

BUTTERWORTH

```
1 from scipy import signal
2
3 sos = signal.butter(N=10, Wn=90, btype='lp',
4                    fs=1000, output='sos')
5 filtered = signal.sosfilt(sos, x3)
6 plt.plot(filtered); plt.xlim(0,650)
```

Con picos cercanos (80 y 100 Hz) pasan residuos y la gráfica se distorsiona; con 20 y 120 Hz la separación es grande y la salida queda casi senoidal pura.



S2

estabilidad: polos dentro del círculo
unitariométodo: espectro → identificar → filtrar →
interpretarscipy.signal · numpy ·
matplotlib